

CSS Object-Fit Property

Complete Guide: Easy English Explanation

Contents

1	Introduction	2
1.1	What is Object-Fit?	2
1.2	Why Use Object-Fit?	2
1.3	The Problem It Solves	2
1.4	Works With	3
2	Core Concept: Object-Fit Values	4
2.1	Value 1: <code>fill</code> (Default)	4
2.1.1	What It Does	4
2.1.2	Visual Result	4
2.1.3	Code Example	4
2.1.4	When to Use <code>fill</code>	4
2.2	Value 2: <code>contain</code>	4
2.2.1	What It Does	4
2.2.2	Visual Result	5
2.2.3	Code Example	5
2.2.4	When to Use <code>contain</code>	5
2.3	Value 3: <code>cover</code>	5
2.3.1	What It Does	5
2.3.2	Visual Result	5
2.3.3	Code Example	5
2.3.4	When to Use <code>cover</code>	6
2.4	Value 4: <code>scale-down</code>	6
2.4.1	What It Does	6
2.4.2	Visual Result	6
2.4.3	Code Example	6
2.4.4	When to Use <code>scale-down</code>	6
2.5	Value 5: <code>none</code>	7
2.5.1	What It Does	7
2.5.2	Visual Result	7
2.5.3	Code Example	7
2.5.4	When to Use <code>none</code>	7
3	Object-Position: Controlling the Crop	8
3.1	What is Object-Position?	8
3.1.1	Default: Center	8
3.1.2	Common Positions	8

3.2	Example: Cropping Face Portraits	8
3.3	Example: Product Images	9
4	Side-by-Side Comparison	10
4.1	All Five Values Compared	10
4.1.1	fill	10
4.1.2	contain	10
4.1.3	cover	10
4.1.4	scale-down	11
4.1.5	none	11
4.2	Comparison Table	11
5	Practical Examples	12
5.1	Example 1: Image Gallery Grid	12
5.2	Example 2: Card Component	12
5.3	Example 3: Hero Section	13
5.4	Example 4: Avatar/Profile Picture	13
5.5	Example 5: Responsive Product Image	14
5.6	Example 6: Video Player Thumbnail	14
5.7	Example 7: Background Image Replacement	15
5.8	Example 8: Aspect Ratio Box	15
5.9	Example 9: Logo Display	15
5.10	Example 10: Responsive Image Grid	16
6	Object-Fit with Video	17
6.1	Video Element	17
6.2	Video with Fallback Image	17
7	Browser Support	19
7.1	Compatibility	19
7.2	Fallback for Older Browsers	19
8	Common Mistakes	20
8.1	Mistake 1: Forgetting overflow: hidden	20
8.2	Mistake 2: Using fill with Images	20
8.3	Mistake 3: Not Setting Explicit Dimensions	21
8.4	Mistake 4: Forgetting object-position	21
8.5	Mistake 5: Using with max-width/max-height Only	22
9	Quick Decision Guide	23
9.1	Choosing the Right Object-Fit Value	23
9.2	Quick Decision Table	23
10	Advanced Tips	24
10.1	Responsive Images with aspect-ratio	24
10.2	Combining with CSS Filters	24
10.3	Object-Fit with Lazy Loading	24

11 Quick Reference	26
11.1 Object-Fit Values Summary	26
11.2 Complete Setup Checklist	26
11.3 Object-Position Values	26
11.4 Common Use Cases	27
11.5 Performance Notes	27
11.6 Related CSS Properties	27
11.7 Browser DevTools Tip	28

Chapter 1

Introduction

1.1 What is Object-Fit?

The `object-fit` property controls how an image or video fills its container. It determines whether the image gets stretched, cropped, scaled, or preserved in its original aspect ratio when the container size doesn't match the image size.

Think of object-fit like framing a photograph:

- **Without frame control:** Photo stretches or squishes to fit the frame
- **With frame control:** Photo stays proportional, and the frame shows what fits best

Object-fit gives you control over how images and videos appear in different sized containers!

1.2 Why Use Object-Fit?

Object-fit is essential for:

- Preventing image distortion and stretching
- Creating responsive image galleries
- Maintaining image aspect ratios
- Building consistent card layouts
- Video player controls
- Thumbnail consistency across different screens
- Professional image presentation
- Creating cropped image effects

1.3 The Problem It Solves

Without object-fit:

```
img {  
  width: 300px;  
  height: 200px;  
  /* Image stretches/squishes to fit - looks distorted! */  
}
```

With object-fit:

```
img {  
  width: 300px;  
  height: 200px;  
  object-fit: cover;  
  /* Image maintains proportions, crops to fit - looks good! */  
}
```

1.4 Works With

Object-fit works on:

- `<img>` elements
- `<video>` elements
- `<canvas>` elements
- Any element with replaced content

Chapter 2

Core Concept: Object-Fit Values

2.1 Value 1: `fill` (Default)

2.1.1 What It Does

The `fill` value stretches or squishes the image to completely fill the container. The image ignores its original aspect ratio and fills the entire width and height of the container.

2.1.2 Visual Result

Image gets stretched/distorted to fill the exact container size.

2.1.3 Code Example

```
img {  
  width: 300px;  
  height: 200px;  
  object-fit: fill; /* or just omit - default */  
}
```

2.1.4 When to Use `fill`

- When distortion is acceptable
- Full coverage is more important than proportions
- Rarely used in modern designs (usually looks bad)
- Background images are a better alternative

2.2 Value 2: `contain`

2.2.1 What It Does

The `contain` value scales the image to fit entirely within the container while maintaining its original aspect ratio. The entire image is always visible, but there may be empty space (letterboxing) on the sides or top/bottom.

2.2.2 Visual Result

Full image visible, maintains proportions, empty space on sides or top/bottom.

2.2.3 Code Example

```
img {  
  width: 300px;  
  height: 200px;  
  object-fit: contain;  
}  
  
/* Image is scaled to fit in 300x200 box  
   Entire image always visible  
   May have empty space (letterbox effect)  
*/
```

2.2.4 When to Use contain

- Logo displays (need full visibility)
- Product images (see entire product)
- Documents and PDFs
- When preserving the complete image is priority
- Avoiding cropping

2.3 Value 3: cover

2.3.1 What It Does

The **cover** value scales the image to completely cover the container while maintaining its original aspect ratio. The image fills the entire container, but parts of it get cropped if necessary. This is the most commonly used value.

2.3.2 Visual Result

Container completely filled, maintains proportions, parts may be cropped.

2.3.3 Code Example

```
img {  
  width: 300px;  
  height: 200px;  
  object-fit: cover;  
}  
  
/* Image is scaled to cover entire 300x200 box  
   Entire container filled  
   Parts of image may be cropped
```

```
*/
```

2.3.4 When to Use `cover`

- Hero image sections (most common use)
- Card image thumbnails
- Background-like effects with `<img>`
- Gallery images
- Video thumbnails
- Ensuring container is completely filled

2.4 Value 4: `scale-down`

2.4.1 What It Does

The `scale-down` value scales the image down to fit the container OR shows the original size, whichever is smaller. It never enlarges images, maintaining or reducing size while preserving aspect ratio.

2.4.2 Visual Result

Image scaled down if needed, or shown at original size (whichever is smaller). No enlargement. Similar to `contain` but won't upscale.

2.4.3 Code Example

```
img {  
  width: 300px;  
  height: 200px;  
  object-fit: scale-down;  
}  
  
/* If original image is smaller than 300x200:  
   Shows at original size  
   If original image is larger:  
   Scales down to fit, maintains aspect ratio  
*/
```

2.4.4 When to Use `scale-down`

- Preventing image upscaling (quality loss)
- Small icon/thumbnail displays
- When you want the smaller of two sizes

- Responsive designs with variable image sizes
- Rare use case, less common than cover/contain

2.5 Value 5: none

2.5.1 What It Does

The **none** value shows the original image at its full, unscaled size. The image is not resized at all. If the image is larger than the container, it will overflow (be cut off).

2.5.2 Visual Result

Original image size, no scaling. May overflow or be partially hidden.

2.5.3 Code Example

```
img {  
  width: 300px;  
  height: 200px;  
  object-fit: none;  
}  
  
/* Image shown at 100% original size  
   Not scaled at all  
   May be clipped if larger than container  
*/
```

2.5.4 When to Use none

- Rarely used in practice
- When you want original size preservation
- Special effects where cropping is intentional
- Usually, other values are better

Chapter 3

Object-Position: Controlling the Crop

3.1 What is Object-Position?

`object-position` works with `object-fit` to control which part of the image shows when cropping occurs (like with `cover`). It positions the image within the container.

3.1.1 Default: Center

```
img {  
  object-fit: cover;  
  object-position: center; /* default */  
  /* Image centered in container, edges cropped */  
}
```

3.1.2 Common Positions

```
/* Horizontal and vertical alignment */  
object-position: left top;  
object-position: center center; /* default */  
object-position: right bottom;  
object-position: 50% 50%; /* percentages */  
object-position: 10px 20px; /* pixel values */  
  
/* Useful combinations */  
object-position: center top; /* Keep faces at top */  
object-position: center center; /* Balanced crop */  
object-position: right center; /* Focus on right side */
```

3.2 Example: Cropping Face Portraits

```
.profile-image {  
  width: 200px;  
  height: 200px;  
  object-fit: cover;  
  object-position: center top;
```

```
/* Crops from bottom, keeps faces visible */  
}
```

3.3 Example: Product Images

```
.product-thumbnail {  
  width: 150px;  
  height: 150px;  
  object-fit: cover;  
  object-position: center center;  
  /* Balanced crop, shows product center */  
}
```

Chapter 4

Side-by-Side Comparison

4.1 All Five Values Compared

Assuming a 300x200px container with a 400x300px image:

4.1.1 fill

```
.image {  
  width: 300px;  
  height: 200px;  
  object-fit: fill;  
}  
  
/* Result: Image stretched to 300x200  
   Aspect ratio: DESTROYED (distorted)  
   Visible: 100% of container, 100% of image  
   Empty space: None  
*/
```

4.1.2 contain

```
.image {  
  width: 300px;  
  height: 200px;  
  object-fit: contain;  
}  
  
/* Result: Image scaled to fit (e.g., 300x225)  
   Aspect ratio: PRESERVED  
   Visible: 100% of image, maybe 89% of container  
   Empty space: Top/bottom letterbox  
*/
```

4.1.3 cover

```
.image {
  width: 300px;
  height: 200px;
  object-fit: cover;
}

/* Result: Image scaled to cover (e.g., 300x200+)
   Aspect ratio: PRESERVED
   Visible: Maybe 100% of container, 88% of image
   Empty space: None (sides may be cropped)
*/
```

4.1.4 scale-down

```
.image {
  width: 300px;
  height: 200px;
  object-fit: scale-down;
}

/* Result: Same as contain or original size
   (whichever is smaller)
   Aspect ratio: PRESERVED
   Visible: 100% of image
   Empty space: Possible letterbox
*/
```

4.1.5 none

```
.image {
  width: 300px;
  height: 200px;
  object-fit: none;
}

/* Result: Original size (not scaled)
   Aspect ratio: PRESERVED
   Visible: Partial (cropped)
   Empty space: Depends on original size
*/
```

4.2 Comparison Table

Value	Aspect Ratio	Image Visible	Crop	Distort
fill	Broken	100%	No	YES
contain	Preserved	100%	No	No
cover	Preserved	Partial	YES	No
scale-down	Preserved	100%	No	No
none	Preserved	Partial	YES	No

Chapter 5

Practical Examples

5.1 Example 1: Image Gallery Grid

```
.gallery {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 20px;
}

.gallery-item {
  width: 200px;
  height: 200px;
  border-radius: 8px;
  overflow: hidden;
}

.gallery-item img {
  width: 100%;
  height: 100%;
  object-fit: cover;
  /* All images same size, consistent appearance */
  /* No distortion, professional look */
}
```

5.2 Example 2: Card Component

```
.card {
  width: 300px;
  border-radius: 12px;
  overflow: hidden;
  box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}

.card-image {
  width: 100%;
  height: 200px;
  background: #f0f0f0;
}
```

```
}

.card-image img {
  width: 100%;
  height: 100%;
  object-fit: cover;
  /* Image fills card, maintains proportion */
}

.card-content {
  padding: 16px;
}
```

5.3 Example 3: Hero Section

```
.hero {
  width: 100%;
  height: 500px;
  position: relative;
  overflow: hidden;
}

.hero img {
  width: 100%;
  height: 100%;
  object-fit: cover;
  object-position: center center;
  /* Background image-like effect */
}

.hero-content {
  position: absolute;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
  color: white;
  text-align: center;
  z-index: 10;
}
```

5.4 Example 4: Avatar/Profile Picture

```
.avatar {
  width: 80px;
  height: 80px;
  border-radius: 50%;
  overflow: hidden;
  border: 3px solid #3498db;
}
```

```
.avatar img {  
  width: 100%;  
  height: 100%;  
  object-fit: cover;  
  object-position: center top;  
  /* Crop from bottom to keep face visible */  
}
```

5.5 Example 5: Responsive Product Image

```
.product-image-container {  
  width: 100%;  
  max-width: 500px;  
  aspect-ratio: 1 / 1;  
  background: #f9f9f9;  
}  
  
.product-image-container img {  
  width: 100%;  
  height: 100%;  
  object-fit: contain;  
  /* Entire product visible, no distortion */  
  padding: 20px;  
  box-sizing: border-box;  
}
```

5.6 Example 6: Video Player Thumbnail

```
.video-thumbnail {  
  width: 320px;  
  height: 180px;  
  position: relative;  
  cursor: pointer;  
  overflow: hidden;  
  border-radius: 4px;  
}  
  
.video-thumbnail img {  
  width: 100%;  
  height: 100%;  
  object-fit: cover;  
  /* Maintains 16:9 aspect ratio */  
  /* Consistent thumbnail size */  
}  
  
.play-button {  
  position: absolute;  
  top: 50%;  
  left: 50%;  
  transform: translate(-50%, -50%);  
}
```

```
width: 60px;
height: 60px;
background: rgba(0,0,0,0.6);
border-radius: 50%;
display: flex;
align-items: center;
justify-content: center;
color: white;
font-size: 24px;
}
```

5.7 Example 7: Background Image Replacement

```
/* Instead of background-image (can't control loading) */
.banner {
  width: 100%;
  height: 300px;
  overflow: hidden;
}

.banner img {
  width: 100%;
  height: 100%;
  object-fit: cover;
  /* Behaves like background-size: cover */
}
```

5.8 Example 8: Aspect Ratio Box

```
/* Maintain aspect ratio responsively */
.aspect-ratio-box {
  width: 100%;
  aspect-ratio: 16 / 9;
  overflow: hidden;
  border-radius: 8px;
}

.aspect-ratio-box img {
  width: 100%;
  height: 100%;
  object-fit: cover;
  /* Fills entire box, maintains quality */
}
```

5.9 Example 9: Logo Display

```
.logo-container {
  width: 150px;
```

```
height: 150px;
background: white;
display: flex;
align-items: center;
justify-content: center;
}

.logo-container img {
max-width: 100%;
max-height: 100%;
object-fit: contain;
/* Full logo visible, never distorted */
}
```

5.10 Example 10: Responsive Image Grid

```
.image-grid {
display: grid;
grid-template-columns: repeat(auto-fill,
                             minmax(250px, 1fr));
gap: 16px;
}

.image-grid-item {
aspect-ratio: 4 / 3;
overflow: hidden;
border-radius: 8px;
background: #e0e0e0;
}

.image-grid-item img {
width: 100%;
height: 100%;
object-fit: cover;
/* Consistent 4:3 ratio, no distortion */
/* Works on any screen size */
}
```

Chapter 6

Object-Fit with Video

6.1 Video Element

Object-fit also works with `<video>` elements:

```
.video-container {  
  width: 100%;  
  height: 600px;  
}  
  
.video-container video {  
  width: 100%;  
  height: 100%;  
  object-fit: cover;  
  /* Video fills container */  
  /* Maintains aspect ratio */  
  /* Prevents distortion */  
}
```

6.2 Video with Fallback Image

```
<div class="video-wrapper">  
  <video  
    width="100%"  
    height="100%"  
    controls  
    poster="/poster-image.jpg"  
  >  
    <source src="video.mp4" type="video/mp4">  
  </video>  
</div>  
  
/* CSS */  
.video-wrapper {  
  width: 100%;  
  height: 400px;  
  overflow: hidden;  
}
```

```
.video-wrapper video {  
  width: 100%;  
  height: 100%;  
  object-fit: cover;  
}
```

Chapter 7

Browser Support

7.1 Compatibility

Object-fit is well-supported in all modern browsers:

- Chrome 32+
- Firefox 36+
- Safari 10+
- Edge 16+
- Opera 19+
- iOS Safari 10+
- Chrome Mobile 32+
- Samsung Internet 6+

7.2 Fallback for Older Browsers

For IE11 and older browsers, use JavaScript or CSS alternatives:

```
/* Fallback with background-image */
.image-fallback {
  background-size: cover;
  background-position: center;
  background-image: url('image.jpg');
}

/* Modern browsers use object-fit */
@supports (object-fit: cover) {
  .image-fallback {
    object-fit: cover;
  }
}
```

Chapter 8

Common Mistakes

8.1 Mistake 1: Forgetting overflow: hidden

Wrong:

```
.container {  
  width: 300px;  
  height: 200px;  
}  
  
.container img {  
  object-fit: cover;  
  /* Image overflows outside container! */  
}
```

Correct:

```
.container {  
  width: 300px;  
  height: 200px;  
  overflow: hidden; /* REQUIRED */  
  border-radius: 8px; /* Optional but common */  
}  
  
.container img {  
  width: 100%;  
  height: 100%;  
  object-fit: cover;  
}
```

8.2 Mistake 2: Using fill with Images

Wrong:

```
img {  
  width: 400px;  
  height: 300px;  
  object-fit: fill;  
  /* Image looks stretched and distorted */  
}
```

Better:

```
img {  
  width: 400px;  
  height: 300px;  
  object-fit: cover; /* or contain */  
  /* Image looks natural and proportional */  
}
```

8.3 Mistake 3: Not Setting Explicit Dimensions

Wrong:

```
img {  
  object-fit: cover;  
  /* No width/height specified */  
  /* Object-fit has no effect! */  
}
```

Correct:

```
img {  
  width: 300px;  
  height: 200px;  
  object-fit: cover;  
  /* Explicit dimensions required */  
}
```

8.4 Mistake 4: Forgetting object-position

Suboptimal:

```
.profile img {  
  width: 200px;  
  height: 200px;  
  object-fit: cover;  
  /* Crops from all sides equally */  
  /* May cut off face! */  
}
```

Better:

```
.profile img {  
  width: 200px;  
  height: 200px;  
  object-fit: cover;  
  object-position: center top;  
  /* Keeps face visible, crops from bottom */  
}
```

8.5 Mistake 5: Using with max-width/max-height Only

Wrong:

```
img {  
  max-width: 300px;  
  max-height: 200px;  
  object-fit: cover;  
  /* Object-fit needs explicit width/height */  
  /* Doesn't work properly */  
}
```

Correct:

```
img {  
  width: 100%;  
  height: 100%;  
  object-fit: cover;  
  /* Within a container with defined dimensions */  
}  
  
.container {  
  max-width: 300px;  
  height: 200px;  
}
```

Chapter 9

Quick Decision Guide

9.1 Choosing the Right Object-Fit Value

1. Do you want to fill the entire container?

- YES → Use `cover`
- NO → Continue to step 2

2. Must the entire image be visible?

- YES → Use `contain`
- NO → Use `cover`

3. Never upscale image?

- YES → Use `scale-down`
- NO → Use `contain` or `cover`

9.2 Quick Decision Table

Use Case	Object-Fit	Notes
Gallery grid	<code>cover</code>	Consistent thumbnails
Product image	<code>contain</code>	Show entire product
Hero section	<code>cover</code>	Background effect
Logo	<code>contain</code>	Never distort logo
Avatar/Profile	<code>cover</code>	Crop with position control
Video thumbnail	<code>cover</code>	16:9 consistency
Card image	<code>cover</code>	Professional look

Chapter 10

Advanced Tips

10.1 Responsive Images with aspect-ratio

```
.image-wrapper {  
  width: 100%;  
  aspect-ratio: 16 / 9;  
  overflow: hidden;  
}  
  
.image-wrapper img {  
  width: 100%;  
  height: 100%;  
  object-fit: cover;  
  /* Maintains 16:9 ratio on any screen */  
}
```

10.2 Combining with CSS Filters

```
.image-container img {  
  width: 100%;  
  height: 100%;  
  object-fit: cover;  
  filter: grayscale(100%);  
  transition: filter 0.3s;  
}  
  
.image-container:hover img {  
  filter: grayscale(0%);  
  /* Color filter effect */  
}
```

10.3 Object-Fit with Lazy Loading

```
img {  
  width: 100%;
```

```
height: 100%;  
object-fit: cover;  
loading: lazy;  
/* Combines lazy loading with object-fit */  
}
```

Chapter 11

Quick Reference

11.1 Object-Fit Values Summary

Value	Behavior	Best For
fill	Stretch to fit	Rarely used
contain	Scale, keep visible	Logos, products
cover	Scale, crop	Most common
scale-down	No upscale	Small images
none	No scaling	Rare

11.2 Complete Setup Checklist

Set container width and height

Add `overflow: hidden` (usually needed)

Set image width: 100% and height: 100%

Choose appropriate object-fit value

Add object-position if cropping matters

Test on different screen sizes

Verify on mobile devices

11.3 Object-Position Values

```
/* Keyword combinations */
object-position: left top;
object-position: center center;
object-position: right bottom;

/* Percentages */
object-position: 50% 50%;      /* Center */
object-position: 0% 50%;      /* Left middle */
object-position: 100% 0%;     /* Top right */
```

```
/* Pixel values */  
object-position: 10px 20px;  
object-position: 50px 50%;    /* Mixed units */
```

11.4 Common Use Cases

- Image galleries (cover)
- Card layouts (cover)
- Hero sections (cover)
- Product displays (contain)
- Avatar/profile pictures (cover + position)
- Video thumbnails (cover)
- Responsive backgrounds (cover)
- Logo displays (contain)
- Blog article images (cover)
- Social media feeds (cover)

11.5 Performance Notes

- Object-fit is GPU-accelerated
- No performance penalty vs CSS alternatives
- Better than stretching images with CSS
- More efficient than background-size equivalents
- No JavaScript needed

11.6 Related CSS Properties

- `object-position` - Control crop position
- `background-size` - Similar for backgrounds
- `aspect-ratio` - Maintain proportions
- `overflow` - Clip overflowing content
- `border-radius` - Rounded corners

11.7 Browser DevTools Tip

In Chrome DevTools:

1. Right-click image
2. Select "Inspect"
3. In Elements tab, hover over `<img>`
4. Look for "object-fit" in computed styles
5. Test different values in real-time